

Class: WebSocketClient

Defined in: [stores/ws.svelte.ts:149](#)

WebSocketClient

Brief

Client WebSocket Stateful e Reattivo. Gestisce riconessioni automatiche (Exponential Backoff), correlazione tra richieste e risposte, e incapsula gli stati della connessione sfruttando Svelte 5 (`$state`).

Tag

FRONT-WS-002

Constructors

Constructor

```
new WebSocketClient(): WebSocketClient
```

Returns

```
WebSocketClient
```

Properties

`_nextRequestId`

```
private _nextRequestId: number = 1
```

Defined in: [stores/ws.svelte.ts:161](#)

`closeHandlers`

```
private closeHandlers: () => void | Promise < void >[] = []
```

Defined in: [stores/ws.svelte.ts:166](#)

Returns

```
void | Promise < void >
```

`connectionStatus`

```
connectionStatus: ConnectionStatus
```

Defined in: [stores/ws.svelte.ts:154](#)

Metadati e stato della connessione (reattivi per interfacciarsi con l'UI).

intentionalClose

```
private intentionalClose: boolean = false
```

Defined in: [stores/ws.svelte.ts:172](#)

onHandlers

```
private onHandlers: Map < string , Set < MessageHandler >>
```

Defined in: [stores/ws.svelte.ts:164](#)

openHandlers

```
private openHandlers: () => void | Promise < void >[] = []
```

Defined in: [stores/ws.svelte.ts:165](#)

Returns

```
void | Promise < void >
```

pendingRequests

```
private pendingRequests: Map < string , PendingRequest >
```

Defined in: [stores/ws.svelte.ts:162](#)

reconnectDelayMs

```
private reconnectDelayMs: number = 1000
```

Defined in: [stores/ws.svelte.ts:171](#)

reconnectTimer

```
private reconnectTimer: number | null = null
```

Defined in: [stores/ws.svelte.ts:170](#)

socket

```
socket: WebSocket | null
```

Defined in: [stores/ws.svelte.ts:151](#)

Istanza nativa del WebSocket (reattiva).

visibilityListenerAttached

```
private visibilityListenerAttached: boolean = false
```

Defined in: [stores/ws.svelte.ts:168](#)

Accessors

isConnected

Get Signature

```
get isConnected(): boolean
```

Defined in: [stores/ws.svelte.ts:177](#)

Brief

Verifica se il WebSocket è attualmente aperto e attivo.

Returns

```
boolean
```

Methods

_connectOnce()

```
private _connectOnce(): Promise < void >
```

Defined in: [stores/ws.svelte.ts:307](#)

Returns

```
Promise < void >
```

Brief

Routine interna per stabilire una singola connessione grezza e legare gli eventi nativi.

_dispatch()

```
private _dispatch( data ): void
```

Defined in: [stores/ws.svelte.ts:356](#)

Parameters

data

```
Record < string , unknown >
```

Returns

```
void
```

Brief

Smista il messaggio in ingresso risolvendo eventuali richieste pendenti o innescando Listener globali.

_scheduleReconnect()

```
private _scheduleReconnect(): void
```

Defined in: [stores/ws.svelte.ts:382](#)

Returns

`void`

Brief

Implementa un algoritmo di Exponential Backoff per tentare le riconessioni. Raddoppia il ritardo ad ogni tentativo fallito fino a un cap massimo.

connect()

connect(): `Promise < void >`

Defined in: [stores/ws.svelte.ts:185](#)

Returns

`Promise < void >`

Promise risolta quando la connessione si apre con successo.

Brief

Installa e stabilisce la connessione WebSocket. Sicuro da chiamare più volte.

disconnect()

disconnect(code , reason): `void`

Defined in: [stores/ws.svelte.ts:198](#)

Parameters

code

`number`

Il codice di chiusura WebSocket.

reason

`string`

La motivazione testuale per la chiusura.

Returns

`void`

Brief

Disconnette intenzionalmente il WebSocket bloccando il sistema di auto-riconnessione.

emit()

emit(action , payload?): `void`

Defined in: [stores/ws.svelte.ts:225](#)

Parameters

action

string

L'azione da compiere (ClientAction).

payload?

Record < string , unknown > = {}

Dati opzionali da allegare in formato JSON.

Returns

void

Brief

Invia un messaggio asincrono al server senza attendere alcuna risposta (Fire and Forget). Non allega nessun `request_id` al pacchetto.

emitAndWait()

emitAndWait(action , payload? , timeoutMs?): Promise < [WsResponse](#) >

Defined in: [stores/ws.svelte.ts:240](#)

Parameters

action

string

L'azione da compiere (ClientAction).

payload?

Record < string , unknown > = {}

Dati opzionali da allegare.

timeoutMs?

number = 5000

Tempo massimo di attesa prima di abortire (Default: 5000ms).

Returns

Promise < [WsResponse](#) >

Promise risolta col pacchetto di risposta del server.

Brief

Invia un messaggio e attende la risposta correlata dal server generando un `request_id`. Risolve in una `WsResponse` anche per errori logici generati dal server, ma rigetta in caso di fallimenti infrastrutturali (es. caduta rete o Timeout).

on()

```
on( action , handler ): () => void
```

Defined in: [stores/ws.svelte.ts:271](#)

Parameters

action

string

L'azione del server da ascoltare (usa "*" per ascoltare tutto).

handler

[MessageHandler](#)

La funzione di callback innescata alla ricezione del messaggio.

Returns

Una lambda per rimuovere l'iscrizione all'evento.

() => void

Brief

Registra un Listener globale per intercettare specifiche Server Action.

onClose()

```
onClose( handler ): () => void
```

Defined in: [stores/ws.svelte.ts:295](#)

Parameters

handler

() => void | Promise < void >

Returns

Funzione per rimuovere l'hook.

() => void

Brief

Registra un Listener innescato alla chiusura del socket.

onOpen()

```
onOpen( handler ): () => void
```

Defined in: [stores/ws.svelte.ts:282](#)

Parameters

handler

() => void | Promise < void >

Returns

Funzione per rimuovere l'hook.

() => `void`

Brief

Registra un Listener innescato immediatamente all'apertura del socket. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / WsResponse

Class: WsResponse

Defined in: [stores/ws.svelte.ts:87](#)

WsResponse

Brief

Wrapper unificato per le risposte dei messaggi WebSocket. Mima l'API HTTP Fetch per fornire un flusso di controllo lineare e prevedibile quando si attendono risposte dal server.

Tag

FRONT-WS-001

Constructors

Constructor

`new WsResponse( rawData ): WsResponse`

Defined in: [stores/ws.svelte.ts:97](#)

Parameters

rawData

`Record < string , unknown >`

Returns

`WsResponse`

Properties

action

`readonly action: string`

Defined in: [stores/ws.svelte.ts:91](#)

Il comando ServerAction ritornato nel pacchetto.

data

`readonly data: Record < string , unknown >`

Defined in: [stores/ws.svelte.ts:95](#)

Il dizionario JSON grezzo coi dati restituiti dal server.

ok

`readonly ok: boolean`

Defined in: [stores/ws.svelte.ts:89](#)

True se l'azione ritornata dal server NON è un errore.

request_id?

`readonly optional request_id?: string`

Defined in: [stores/ws.svelte.ts:93](#)

L'ID univoco della richiesta originaria inoltrata dal client, se presente.

Accessors

reason

Get Signature

`get reason(): string`

Defined in: [stores/ws.svelte.ts:109](#)

Brief

Estrae in sicurezza il motivo dell'errore (reason) restituito dal server.

Returns

`string`

Il messaggio di errore o un fallback predefinito.

Methods

get()

`get< T >( key ): T | null`

Defined in: [stores/ws.svelte.ts:118](#)

Type Parameters

T

`T`

Parameters

key

string

La chiave da cercare nel dizionario JSON.

Returns

T | null

Il valore tipizzato o null se la chiave non esiste.

Brief

Estrae un valore tipizzato dal payload.

getOr()

```
getOr< T >( key , fallback ): T
```

Defined in: [stores/ws.svelte.ts:127](#)

Type Parameters

T

T

Parameters

key

string

La chiave da cercare nel payload JSON.

fallback

T

Il valore di sicurezza da restituire.

Returns

T

Brief

Estrae un valore tipizzato dal payload fornendo un fallback in caso di assenza. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / BotTakeoverMode

Enumeration: BotTakeoverMode

Defined in: [stores/lobby.svelte.ts:15](#)

BotTakeoverMode

Brief

Modalità di comportamento quando un bot sostituisce o viene sostituito.

Enumeration Members

PlayInstantly

PlayInstantly: 0

Defined in: [stores/lobby.svelte.ts:17](#)

Il bot gioca istantaneamente la sua mossa non appena arriva il suo turno.

WaitUntilTurnEnd

WaitUntilTurnEnd: 1

Defined in: [stores/lobby.svelte.ts:19](#)

Il bot attende fino a 5 secondi per il suo turno, o fino alla fine del turno per altri giocatori AFK [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / fromResponse

Function: fromResponse()

fromResponse(*res*): *Promise* < [AppError](#) >

Defined in: [utils/errors.ts:33](#)

Build an AppError from a failed fetch Response. Prefers the server's own `error` string if the body is JSON.

Parameters

res

Response

Returns

Promise < [AppError](#) > [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / fromThrown

Function: fromThrown()

fromThrown(*err*): [AppError](#)

Defined in: [utils/errors.ts:55](#)

Wrap any thrown value into an `AppError`.

Parameters

err

unknown

Returns

[AppError](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validateEmail

Function: validateEmail()

validateEmail(*email*): [ValidationResult](#)

Defined in: [utils/validation.ts:30](#)

Validates an email string.

Parameters

email

string

Returns

[ValidationResult](#)

[ValidationResult](#) whether the email is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validateLobbyName

Function: validateLobbyName()

validateLobbyName(*name*): [ValidationResult](#)

Defined in: [utils/validation.ts:84](#)

Validates a lobby string.

Parameters

name

string

Returns

[ValidationResult](#)

ValidationResult whether the lobby is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validatePasswordMatch

Function: validatePasswordMatch()

```
validatePasswordMatch( password , confirm ): ValidationResult
```

Defined in: [utils/validation.ts:73](#)

Validates whether two password strings match.

Parameters

password

string

confirm

string

Returns

[ValidationResult](#)

ValidationResult whether they match, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validatePassword

Function: validatePassword()

```
validatePassword( password ): ValidationResult
```

Defined in: [utils/validation.ts:41](#)

Validates a password string.

Parameters

password

string

Returns

[ValidationResult](#)

ValidationResult whether the password is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validateUsername

Function: validateUsername()

validateUsername(*username*): [ValidationResult](#)

Defined in: [utils/validation.ts:55](#)

Validates a username string.

Parameters

username

string

Returns

[ValidationResult](#)

ValidationResult whether the username is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / AppError

Interface: AppError

Defined in: [utils/errors.ts:10](#)

The backend uses HTTP status codes consistently and returns plain English strings in the `error` JSON field — it does NOT return machine-readable error code strings like "INVALID_PASSWORD".

Use HTTP status codes for routing and fallback messages, and prefer showing the server's own error string when available.

Properties

code

code: string

Defined in: [utils/errors.ts:11](#)

details?

optional details?: Record < string , unknown >

Defined in: [utils/errors.ts:14](#)

message

message: `string`

Defined in: [utils/errors.ts:12](#)

userMessage

userMessage: `string`

Defined in: [utils/errors.ts:13](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / AuthFieldErrors

Interface: AuthFieldErrors

Defined in: [stores/auth.svelte.ts:21](#)

AuthFieldErrors

Brief

Mappatura degli errori per ogni singolo campo del form di registrazione/login.

Properties

confirmPassword?

optional **confirmPassword?:** `string`

Defined in: [stores/auth.svelte.ts:25](#)

email?

optional **email?:** `string`

Defined in: [stores/auth.svelte.ts:23](#)

password?

optional **password?:** `string`

Defined in: [stores/auth.svelte.ts:24](#)

username?

optional **username?:** `string`

Defined in: [stores/auth.svelte.ts:22](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / Card

Interface: Card

Defined in: [stores/game.svelte.ts:37](#)

Card

Brief

Rappresentazione frontend di una singola carta.

Properties

color

color: "red" | "blue" | "green" | "yellow" | "wild"

Defined in: [stores/game.svelte.ts:41](#)

Il colore decodificato della carta.

id

id: number

Defined in: [stores/game.svelte.ts:39](#)

L'identificativo univoco della carta a 16-bit.

value

value: "reverse" | "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" |
"skip" | "+2" | "colorswitch" | "+4"

Defined in: [stores/game.svelte.ts:43](#)

Il valore o l'azione associata alla carta. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ConnectionStatus

Interface: ConnectionStatus

Defined in: [stores/ws.svelte.ts:69](#)

ConnectionStatus

Brief

Rappresenta lo stato reattivo della connessione WebSocket utile per la UI.

Properties

lobby_code

lobby_code: `string`

Defined in: [stores/ws.svelte.ts:77](#)

Codice invito della lobby attuale.

room

room: `string`

Defined in: [stores/ws.svelte.ts:75](#)

Nome della stanza corrente.

status

status: `"connected" | "connecting" | "disconnected"`

Defined in: [stores/ws.svelte.ts:71](#)

Stato attuale della connessione.

username

username: `string`

Defined in: [stores/ws.svelte.ts:73](#)

Username sincronizzato dal server. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / `GamePlayer`

Interface: GamePlayer

Defined in: [stores/game.svelte.ts:51](#)

`GamePlayer`

Brief

Dati associati ad un giocatore in-game. Se il giocatore non è il client corrente, l'array `hand` sarà undefined e sostituito da `card_count` calcolato dal server.

Properties

card_count

card_count: `number`

Defined in: [stores/game.svelte.ts:55](#)

Numero totale di carte in mano al giocatore.

hand?

optional **hand?:** [Card](#) []

Defined in: [stores/game.svelte.ts:59](#)

Le carte fisiche in mano al giocatore (Presente solo per il Local Player).

has_called_uno

has_called_uno: *boolean*

Defined in: [stores/game.svelte.ts:57](#)

Indica se il giocatore ha cliccato correttamente su "UNO!".

is_bot

is_bot: *boolean*

Defined in: [stores/game.svelte.ts:61](#)

Indica se il giocatore è un bot controllato dal server.

username

username: *string*

Defined in: [stores/game.svelte.ts:53](#)

Username del giocatore. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / GameState

Interface: GameState

Defined in: [stores/game.svelte.ts:68](#)

GameState

Brief

Fotografia dello stato completo del tavolo in un preciso istante.

Properties

active_color

active_color: *string*

Defined in: [stores/game.svelte.ts:70](#)

Colore attualmente attivo per le giocate.

current_turn

current_turn: `string`

Defined in: [stores/game.svelte.ts:72](#)

Username del giocatore a cui spetta il turno corrente.

is_over?

optional is_over?: `boolean`

Defined in: [stores/game.svelte.ts:80](#)

Flag che indica se la partita ha raggiunto uno stato terminale.

play_direction

play_direction: `number`

Defined in: [stores/game.svelte.ts:74](#)

Direzione del gioco (1 per orario, -1 per antiorario).

players

players: [GamePlayer](#) []

Defined in: [stores/game.svelte.ts:78](#)

La lista di tutti i giocatori presenti nella partita.

top_card?

optional top_card?: [Card](#)

Defined in: [stores/game.svelte.ts:76](#)

La carta attualmente in cima alla pila degli scarti.

winner?

optional winner?: `string`

Defined in: [stores/game.svelte.ts:82](#)

Username del giocatore vincitore, qualora la partita sia finita. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ListedLobby

Interface: ListedLobby

Defined in: [stores/lobby.svelte.ts:120](#)

ListedLobby

Brief

Dati minimizzati inviati dal server per renderizzare la lista delle lobby pubbliche. Risparmia banda non inviando la lista completa dei membri e le regole dettagliate.

Properties

bot_count

bot_count: *number*

Defined in: [stores/lobby.svelte.ts:126](#)

Numero totale di bot presenti in questa lobby.

invite_code

invite_code: *string*

Defined in: [stores/lobby.svelte.ts:128](#)

Il codice di invito per permettere al client di unirsi con un click.

member_count

member_count: *number*

Defined in: [stores/lobby.svelte.ts:124](#)

Numero totale di giocatori umani attualmente connessi.

name

name: *string*

Defined in: [stores/lobby.svelte.ts:122](#)

Il nome personalizzato della lobby. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / Lobby

Interface: Lobby

Defined in: [stores/lobby.svelte.ts:100](#)

Lobby

Brief

Rappresenta una lobby di gioco completa con tutti i suoi dati di stato.

Properties

host

host: `string`

Defined in: [stores/lobby.svelte.ts:104](#)

L'username del creatore (Host) della lobby.

invite_code

invite_code: `string`

Defined in: [stores/lobby.svelte.ts:102](#)

Il codice univoco alfanumerico di 6 caratteri utilizzato per unirsi.

member_count

member_count: `number`

Defined in: [stores/lobby.svelte.ts:108](#)

Il numero attuale di membri presenti nella lobby (massimo 4).

members

members: [LobbyMember](#) []

Defined in: [stores/lobby.svelte.ts:110](#)

Lista dettagliata dei giocatori attualmente nella lobby.

name

name: `string`

Defined in: [stores/lobby.svelte.ts:106](#)

Il nome personalizzato della lobby mostrato nella lista pubblica.

settings

settings: [LobbySettings](#)

Defined in: [stores/lobby.svelte.ts:112](#)

Le impostazioni configurate per questa stanza di gioco. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / LobbyMember

Interface: LobbyMember

Defined in: [stores/lobby.svelte.ts:85](#)

LobbyMember

Brief

Rappresenta un singolo giocatore connesso all'interno di una lobby.

Properties

is_bot

is_bot: *boolean*

Defined in: [stores/lobby.svelte.ts:93](#)

Vero se questo "membro" è in realtà un account bot gestito dal server.

is_connected

is_connected: *boolean*

Defined in: [stores/lobby.svelte.ts:89](#)

Indica se il giocatore è attualmente connesso tramite WebSocket o è in fase di riconnessione.

is_host

is_host: *boolean*

Defined in: [stores/lobby.svelte.ts:91](#)

Vero se questo giocatore è il proprietario/host della lobby e può avviare la partita.

username

username: *string*

Defined in: [stores/lobby.svelte.ts:87](#)

Il nome visualizzato del giocatore. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / LobbySettings

Interface: LobbySettings

Defined in: [stores/lobby.svelte.ts:40](#)

LobbySettings

Brief

Rappresenta la configurazione e le regole scelte per la lobby. Definisce i parametri della partita e la composizione del mazzo iniziale.

Properties

active_mods

active_mods: `string []`

Defined in: [stores/lobby.svelte.ts:44](#)

Array di regole speciali (Mod) attivate per questa partita.

allow_bot_replacement

allow_bot_replacement: `boolean`

Defined in: [stores/lobby.svelte.ts:63](#)

Se true, un giocatore umano che abbandona viene rimpiazzato da un bot anziché terminare il gioco.

allow_bot_takeover

allow_bot_takeover: `boolean`

Defined in: [stores/lobby.svelte.ts:61](#)

Se true, i nuovi giocatori umani possono entrare in partita sostituendo un bot.

bot_count

bot_count: `number`

Defined in: [stores/lobby.svelte.ts:57](#)

Numero di bot da inserire automaticamente all'avvio della partita.

bot_mode

bot_mode: `BotTakeoverMode`

Defined in: [stores/lobby.svelte.ts:59](#)

Comportamento del bot (es. gioca subito o attende il timer).

count_draw_two

count_draw_two: `number`

Defined in: [stores/lobby.svelte.ts:74](#)

Copie per colore della carta "Pesca Due (+2)".

count_numbered

count_numbered: *number*

Defined in: [stores/lobby.svelte.ts:68](#)

Copie per colore dei numeri da 1 a 9 nel mazzo.

count_reverses

count_reverses: *number*

Defined in: [stores/lobby.svelte.ts:72](#)

Copie per colore della carta "Inverti Giro".

count_skips

count_skips: *number*

Defined in: [stores/lobby.svelte.ts:70](#)

Copie per colore della carta "Divieto/Salta Turno".

count_wild

count_wild: *number*

Defined in: [stores/lobby.svelte.ts:76](#)

Numero totale di carte Jolly (Cambio Colore) nel mazzo.

count_wild_draw_four

count_wild_draw_four: *number*

Defined in: [stores/lobby.svelte.ts:78](#)

Numero totale di carte Jolly Pesca Quattro (+4) nel mazzo.

count_zeros

count_zeros: *number*

Defined in: [stores/lobby.svelte.ts:66](#)

Copie per colore del numero 0 nel mazzo.

is_public

is_public: *boolean*

Defined in: [stores/lobby.svelte.ts:42](#)

Se true, la lobby apparirà nella lista pubblica delle partite.

quit_deletes_match

quit_deletes_match: *boolean*

Defined in: [stores/lobby.svelte.ts:51](#)

Se true, l'abbandono volontario di un giocatore eliminerà definitivamente il salvataggio.

save_state

save_state: *boolean*

Defined in: [stores/lobby.svelte.ts:49](#)

Se true, lo stato della partita verrà salvato nel database ad ogni mossa.

starting_cards

starting_cards: *number*

Defined in: [stores/lobby.svelte.ts:54](#)

Numero di carte distribuite a ciascun giocatore all'inizio della partita.

turn_time_limit_ms

turn_time_limit_ms: *number*

Defined in: [stores/lobby.svelte.ts:46](#)

Limite di tempo in millisecondi concesso per completare un turno (es. 15000). [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / SavedMatch

Interface: SavedMatch

Defined in: [stores/lobby.svelte.ts:26](#)

SavedMatch

Brief

Dati di base di una partita salvata sul server.

Properties

match_id

match_id: *string*

Defined in: [stores/lobby.svelte.ts:28](#)

Identificativo univoco del salvataggio nel database.

players

players: `string []`

Defined in: [stores/lobby.svelte.ts:32](#)

Lista degli username dei giocatori presenti in quella partita.

saved_at

saved_at: `string`

Defined in: [stores/lobby.svelte.ts:30](#)

Data e ora del salvataggio. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / Toast

Interface: Toast

Defined in: [stores/toast.svelte.ts:18](#)

Toast

Brief

Rappresenta una singola notifica attualmente visibile a schermo.

Properties

id

id: `number`

Defined in: [stores/toast.svelte.ts:20](#)

Identificativo numerico univoco della notifica, usato per la rimozione.

message

message: `string`

Defined in: [stores/toast.svelte.ts:24](#)

Il messaggio testuale da mostrare all'utente.

type

type: [ToastType](#)

Defined in: [stores/toast.svelte.ts:22](#)

Tipo di notifica (determina il colore di sfondo/bordo). [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ValidationResult

Interface: ValidationResult

Defined in: [utils/validation.ts:13](#)

Rules here must stay in sync with the C++ backend constraints: kMinUsernameLen = 3, kMaxUsernameLen = 32 (auth_controller.hpp) kMinPasswordLen = 8 (auth_controller.hpp) topic max 64 chars, alphanumeric/-/_ (webserver.cpp)

The backend does NOT require uppercase/lowercase/numbers in passwords — the previous validatePassword rule was stricter than the server and would reject valid passwords. Removed. If you later add server-side complexity rules, add them here at the same time.

Properties

error?

optional **error?**: *string*

Defined in: [utils/validation.ts:15](#)

valid

valid: *boolean*

Defined in: [utils/validation.ts:14](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / AppScreen

Type Alias: AppScreen

AppScreen = *"main" | "auth" | "lobbies" | "lobby" | "game" | "settings" | "stats" | "detailedStats"*

Defined in: [stores/navigation.svelte.ts:14](#)

Brief

Elenco delle schermate disponibili nell'applicazione frontend. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / CardColor

Type Alias: CardColor

CardColor = *typeof COLOR_MAP [number]*

Defined in: [stores/game.svelte.ts:30](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / CardValue

Type Alias: CardValue

```
CardValue = typeof VALUE_MAP [ number ]
```

Defined in: [stores/game.svelte.ts:31](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ClientActionType

Type Alias: ClientActionType

```
ClientActionType = typeof ClientAction [keyof *typeof* ClientAction ]
```

Defined in: [stores/ws.svelte.ts:54](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / MessageHandler

Type Alias: MessageHandler

```
MessageHandler = ( data ) => void
```

Defined in: [stores/ws.svelte.ts:63](#)

Parameters

data

```
Record < string , unknown >
```

Returns

```
void
```

Brief

Firma della funzione di callback per intercettare i messaggi WebSocket. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ServerActionDef

Type Alias: ServerActionDef

```
ServerActionDef = ServerActionType | string
```

Defined in: [stores/ws.svelte.ts:57](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ServerActionType

Type Alias: ServerActionType

ServerActionType = *typeof* [ServerAction](#) [*keyof* **typeof** [ServerAction](#)]

Defined in: [stores/ws.svelte.ts:55](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ToastType

Type Alias: ToastType

ToastType = "success" | "error" | "info" | "warning"

Defined in: [stores/toast.svelte.ts:12](#)

Brief

Tipologia della notifica, utilizzata per determinare lo stile e l'icona del toast. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ClientAction

Variable: ClientAction

const **ClientAction**: *object*

Defined in: [stores/ws.svelte.ts:11](#)

Type Declaration

ChatSend

readonly **ChatSend**: "chat_send" = "chat_send"

GameCallUno

readonly **GameCallUno**: "game_call_uno" = "game_call_uno"

GameDrawCard

readonly **GameDrawCard**: "game_draw_card" = "game_draw_card"

GameExit

readonly **GameExit**: "game_exit" = "game_exit"

GamePlayCard

readonly **GamePlayCard**: "game_play_card" = "game_play_card"

GameSubmitInput

```
readonly GameSubmitInput: "game_submit_input" = "game_submit_input"
```

LobbyCreate

```
readonly LobbyCreate: "lobby_create" = "lobby_create"
```

LobbyDeleteSavedMatch

```
readonly LobbyDeleteSavedMatch: "lobby_delete_saved_match" =  
"lobby_delete_saved_match"
```

LobbyJoin

```
readonly LobbyJoin: "lobby_join" = "lobby_join"
```

LobbyKick

```
readonly LobbyKick: "lobby_kick" = "lobby_kick"
```

LobbyLeave

```
readonly LobbyLeave: "lobby_leave" = "lobby_leave"
```

LobbyList

```
readonly LobbyList: "lobby_list" = "lobby_list"
```

LobbyListSavedMatches

```
readonly LobbyListSavedMatches: "lobby_list_saved_matches" =  
"lobby_list_saved_matches"
```

LobbyPromote

```
readonly LobbyPromote: "lobby_promote" = "lobby_promote"
```

LobbyRejoin

```
readonly LobbyRejoin: "lobby_rejoin" = "lobby_rejoin"
```

LobbyResumeSavedMatch

```
readonly LobbyResumeSavedMatch: "lobby_resume_saved_match" =  
"lobby_resume_saved_match"
```

LobbyStartMatch

```
readonly LobbyStartMatch: "lobby_start_match" = "lobby_start_match"
```

LobbyUpdateSettings

```
readonly LobbyUpdateSettings: "lobby_update_settings" = "lobby_update_settings"
```

Brief

Dizionario delle costanti per le azioni inviate dal Client verso il Server. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ServerAction

Variable: ServerAction

```
const ServerAction: object
```

Defined in: [stores/ws.svelte.ts:38](#)

Type Declaration

ChatMessage

```
readonly ChatMessage: "chat_message" = "chat_message"
```

Error

```
readonly Error: "error" = "error"
```

GameOver

```
readonly GameOver: "game_over" = "game_over"
```

GameStateUpdated

```
readonly GameStateUpdated: "game_state_updated" = "game_state_updated"
```

LobbyEvicted

```
readonly LobbyEvicted: "lobby_evicted" = "lobby_evicted"
```

LobbyJoined

```
readonly LobbyJoined: "lobby_joined" = "lobby_joined"
```

LobbyLeft

```
readonly LobbyLeft: "lobby_left" = "lobby_left"
```

LobbyList

```
readonly LobbyList: "lobby_list" = "lobby_list"
```

LobbyUpdated

```
readonly LobbyUpdated: "lobby_updated" = "lobby_updated"
```

Success

```
readonly Success: "success" = "success"
```

Brief

Dizionario delle costanti per le azioni inviate dal Server verso il Client. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeAuth

Variable: storeAuth

```
const storeAuth: StoreAuth
```

Defined in: [stores/auth.svelte.ts:214](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeGame

Variable: storeGame

```
const storeGame: StoreGame
```

Defined in: [stores/game.svelte.ts:261](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeLobby

Variable: storeLobby

```
const storeLobby: StoreLobby
```

Defined in: [stores/lobby.svelte.ts:411](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeNavigation

Variable: storeNavigation

```
const storeNavigation: StoreNavigation
```

Defined in: [stores/navigation.svelte.ts:71](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeToast

Variable: storeToast

```
const storeToast: StoreToast
```

Defined in: [stores/toast.svelte.ts:104](#)

Brief

Istanza singleton esportata globalmente. È il punto di accesso per tutti gli altri store. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ws

Variable: ws

```
const ws: WebSocketClient
```

Defined in: [stores/ws.svelte.ts:423](#)